



**UNIVERSITÀ
DI TORINO**

UNIVERSITÀ DEGLI STUDI DI TORINO

Corso di Laurea Magistrale in Fisica dei Sistemi Complessi

Improving VAE Representational Capabilities for Understanding LLMs

Tesi di Laurea Magistrale

Relatore:

Prof. Michele Caselle

Corelatori:

Prof. Alan Perotti

Prof. André Panisson

Controrelatore:

Prof. Matteo Osella

Candidato:

Francesco Marchisotti

Tutor Aziendale:

Gabriele Ciravegna

Anno Accademico 2025/2026

Francesco Marchisotti: *Improving VAE Representational Capabilities for Understanding LLMs*, Fisica dei Sistemi Complessi

CENTAI TEAM:

Gabriele Ciravegna

Alan Perotti

André Panisson

Francesco Cozzi

UNITO PROFESSORS:

Michele Caselle

Matteo Osella

LOCATION:

Torino

TIME FRAME:

Oct. 2025 - Jul. 2026

“I wish there was a way to know you’re in the good old days
before you’ve actually left them.”

— *Andy Bernard*

ABSTRACT

Large Language Models encode their computations in dense, high-dimensional activations whose individual dimensions seldom correspond to anything a human can name, and Sparse AutoEncoders (SAEs) have become the standard tool for decomposing these activations into interpretable concepts. Yet SAEs impose a steep sparsity–reconstruction trade-off: because each concept is a single scalar coefficient, holding the reconstruction error at an acceptable level leaves far too many active concepts per token for the decomposition to be interpretable. This thesis introduces the Variational Auto Embedding Encoder (VAEE), a model that represents each concept not as a scalar but as a learned vector embedding, drawing together ideas from Concept Embedding Models, Variational AutoEncoders, and Vector-Quantized Variational AutoEncoders (VQ-VAEs); like a VQ-VAE it decodes through a learned codebook, but rather than reconstructing from a single embedding it stacks every embedding deemed active — each selected by comparing the encoded input against a learned prototype — and decodes from that stack. Derived from first principles, the VAEE markedly relaxes the sparsity–reconstruction trade-off: on Mistral 7B residual-stream activations, a VAEE with eight active concepts matches the reconstruction of a parameter-matched TopK SAE that requires 128. These gains come with honest caveats: reconstruction was assessed only by mean squared error, each concept now spans several active neurons rather than one, and the recovered concepts were not found to be more interpretable than the SAE’s — so the VAEE’s contribution lies in expanding the capacity and sparsity of concept representations, leaving their interpretability and downstream steering as the central open questions.

ACKNOWLEDGMENTS

To whom shall be acknowledged,
you are being acknowledged with this exact statement.

“I’m so funny.”

— *The candidate, who’s not actually funny*

CONTENTS

1	INTRODUCTION	1
1.1	Motivation	1
1.2	Thesis Structure	1
2	BACKGROUND AND RELATED WORK	3
2.1	Natural Language Processing	3
2.2	eXplainable AI	3
2.2.1	Traditional XAI	3
2.2.2	Concept-based XAI	4
2.3	AutoEncoders	7
2.3.1	Standard AutoEncoders	7
2.3.2	Variational AutoEncoders	9
2.3.3	Sparse AutoEncoders	12
3	METHODS	19
3.1	Definitions/Notation	19
3.2	Aim of the Work	19
3.3	Model Development	19
3.3.1	Probabilistic Formulation	19
3.3.2	Architectures	19
4	EXPERIMENTS	21
4.1	Model Training	21
4.2	Evaluation Metrics	21
4.2.1	MSE	21
4.2.2	ℓ_0	21
4.2.3	Perplexity	21
4.3	Results	21
5	CONCLUSIONS	23
5.1	Contributions	23
5.2	Future Work	23
	BIBLIOGRAPHY	25

LIST OF FIGURES

Figure 2.1	Illustration of Denoising autoencoder model architecture, reproduced from Weng [46]. <i>Note: this figure uses f for the decoder and g for the encoder, the reverse of the letter convention adopted in this thesis.</i>	8
Figure 2.2	Two-dimensional latent spaces learned on MNIST, coloured by digit class, reproduced from Shafkat [37]. Optimising for reconstruction alone (left) lets a standard autoencoder separate the classes into disconnected clusters with gaps that decode to unrealistic samples, whereas the Variational AutoEncoder objective (right) yields a smoother, more continuous arrangement, concentrated around the origin by the prior, that can be sampled and interpolated.	9
Figure 2.3	Neuron 4e:55, a polysemantic neuron from InceptionV1 which responds to cat faces, fronts of cars, and cat legs. Reproduced from Olah et al. [27].	13

LIST OF TABLES

LISTINGS

ACRONYMS

CB-LLM Concept Bottleneck LLM	6
CBM Concept Bottleneck Model	5
ELBO Evidence Lower Bound	10 , 12
KL Kullback–Leibler	10 ff.
LF-CBM Label-free CBM	5 f.
LLM Large Language Model	3 , 5 ff., 16
MECHINT Mechanistic Interpretability	6 , 13 , 16
MLP Multi-Layer Perceptron	16
MSE Mean Squared Error	7 , 10
PCA Principal Component Analysis	8
SAE Sparse AutoEncoder	6 , 12–17
VAE Variational AutoEncoder	9–12 , 15

INTRODUCTION

1.1 MOTIVATION

1.2 THESIS STRUCTURE

BACKGROUND AND RELATED WORK

2.1 NATURAL LANGUAGE PROCESSING

2.2 EXPLAINABLE AI

The incredible performance of modern machine-learning systems comes at the cost of transparency: a deep network’s output is the product of millions of interacting parameters, with no accompanying account of why it was produced. eXplainable Artificial Intelligence (XAI) is the field that sets out to supply such accounts, making a model’s behaviour intelligible enough to be trusted, debugged, or learnt from. The methods relevant to this thesis are most naturally organised around a shift in what counts as an explanation: from *attribution*, which asks *where* (“which input features drove a prediction?”), to *concepts*, the human-meaningful units in terms of which a model can be said to reason. It is this latter, concept-based view on which this work builds, in the service of understanding Large Language Models (LLMs).

2.2.1 TRADITIONAL XAI

2.2.1.1 *Intrinsically Interpretable Models*

The most direct route to an interpretable model is to choose one whose decision process is transparent by construction. Linear and logistic regression, decision trees, naive Bayes classifiers, and k -nearest neighbours are the canonical examples: their parameters or structure can be inspected directly, so that the explanation *is* the model rather than a separate artifact derived from it [25]. Rudin [34] argues that such models should be preferred wherever they are competitive, since post-hoc explanations of an opaque model are approximations that need not be faithful to its actual computation. This route is, however, closed for the systems studied in this thesis: LLMs comprise billions of nonlinearly interacting parameters, and no direct reading of their weights yields a human-understandable account of their behaviour. Interpretability must therefore be recovered *post hoc*, from a model that is already trained and fixed.

2.2.1.2 *Post-hoc Attribution Methods*

The dominant post-hoc paradigm explains an individual prediction by *attributing* it to the input features, assigning each feature a score that quantifies its contribution to the output. LIME [32] does this by fitting a simple, interpretable surrogate (typically a sparse linear model) to the behaviour of the black box in a local neighbourhood of the instance being explained. SHAP [23] unifies this and several related schemes under the Shapley value from cooperative game theory, yielding feature attributions that are additive and satisfy a set of desirable axioms. For models operating on images, gradient-based saliency maps [38] and their refinements such as Grad-CAM [36] produce a heatmap over the input that highlights the regions most influential to the prediction. What these methods share is also their central limitation: they indicate *where* in the input the model is sensitive, but not *what* the model has internally come to represent. An attribution map can mark a pixel or a token as important without revealing the human-meaningful concept it participates in: a gap that motivates the concept-based methods of section 2.2.2.

COUNTERFACTUAL EXPLANATIONS A counterfactual explanation answers a contrastive question: what is the smallest change to the input that would have produced a different, desired prediction [45]? Rather than scoring the features of the present input, it points to a nearby instance on the other side of the decision boundary: “had this applicant’s income been slightly higher, the loan would have been granted”. Such explanations are attractive because they are actionable and require no access to the model’s internals, only the ability to query it. Generating counterfactuals that are at the same time close to the original input and realistic is itself a modelling problem, one which generative models are naturally suited to solve.

2.2.1.3 *Prototypes*

A complementary, example-based strategy explains a model not through feature scores but through representative instances. A *prototype* is a data point that is typical of a region of the input space, chosen so that a small set of such points summarises the data (or the model’s behaviour upon it) in terms a human can grasp by inspection [17]. The defining characteristic, for the purposes of this thesis, is that a prototype in this traditional sense is an *input instance*: an actual or representative example drawn from the data space.

2.2.2 CONCEPT-BASED XAI

Concept-based XAI answers the *what* question that attribution leaves open: instead of pointing to the inputs responsible for a prediction,

it explains the prediction in terms of high-level, human-meaningful *concepts* (“striped”, “has wheels”, “expresses uncertainty”) of the kind a person would use to describe the same decision. Methods in this family differ chiefly in whether the concepts are built into the model from the outset or recovered from a model that was trained without them.

2.2.2.1 *Explainable-by-design Models*

One way to guarantee that concepts genuinely take part in a model’s decision is to make them an explicit, intermediate stage of the computation, so that the prediction is forced to pass through a concept representation before reaching the output.

CONCEPT BOTTLENECK MODELS Concept Bottleneck Models (CBMs) [20] insert exactly such a stage: the network first maps the input to a set of human-specified concepts and then predicts the label from those concepts alone. Because the output depends on the input only through the concept layer, the model’s reasoning can be read off the activated concepts, and a user can intervene on them, correcting a mistaken concept and observing the effect on the prediction. The downside is supervision: training a CBM requires data annotated by human experts with concept labels, which is costly and ties the model to a fixed, predefined vocabulary.

CONCEPT EMBEDDING MODELS A known weakness of CBMs is an accuracy–interpretability trade-off: when the predefined concepts do not carry all the information the task requires, forcing the prediction through a narrow concept bottleneck degrades performance. Concept Embedding Models [9] loosen the bottleneck by representing each concept not as a single scalar but as a pair of high-dimensional embeddings, one for its active and one for its inactive state. This recovers accuracy comparable to an unconstrained black box while retaining concept-level interpretability and the possibility of intervention. Like CBMs, these models require a concept-annotated dataset.

LABEL-FREE CONCEPT BOTTLENECK MODELS Label-free CBMs (LF-CBMs) [26] address the annotation cost directly. Rather than relying on a hand-labelled concept set, they extract candidate concepts from an LLM and align the network’s activations with them through a vision–language model such as CLIP, converting a standard backbone into a CBM without any concept supervision. This makes the approach scalable to settings where dense concept annotation would be impractical.

CONCEPT BOTTLENECK LLMS The concept-bottleneck idea has recently been carried over to language. Concept Bottleneck LLMs

(CB-LLMs) [39] insert a concept bottleneck layer into an LLM, routing text classification — and, in a generative variant, generation itself — through a set of interpretable concepts before the output. As in an LF-CBM, a hand-annotated concept set is not needed: the concepts are generated by a prompted LLM and each training text is scored against them automatically, so the bottleneck is built without concept supervision. The explicit concept layer also makes the model *steerable*, since intervening on a concept predictably shifts its behaviour. Like the other methods in this group, CB-LLMs are *explainable by design*: interpretability is secured by training the model with the concept bottleneck built in.

Even so, that interpretability is only partial. In the generative variant the next-token prediction is shaped not by the concept bottleneck alone but by the bottleneck together with a parallel layer of unsupervised, uninterpretable neurons added to preserve generation quality. When the dataset labels are reused as concepts the bottleneck can be tiny — four neurons for the four AGnews classes — yet in the authors’ implementation it runs alongside 768 unsupervised ones,¹ so only 4 of the 772 dimensions feeding each token prediction carry interpretable concepts.

2.2.2.2 Post-hoc Methods

The alternative to *explainable-by-design* models is to leave the trained model untouched and probe it for the concepts it has already come to represent.

TESTING WITH CONCEPT ACTIVATION VECTORS T-CAV [18] represents a concept as a direction in a layer’s activation space: given example inputs exhibiting the concept and a set of random ones, it fits a linear classifier to separate them and takes the normal to the decision boundary — the concept activation vector — as the concept’s direction. The model’s sensitivity to the concept is then quantified by the directional derivative of the output along this vector, yielding a global statement of how much a given concept influences a class prediction. The concept must still be supplied in advance, as a set of examples.

SAES AND MECHANISTIC INTERPRETABILITY A final, unsupervised strand removes even that requirement. Rather than testing for concepts named ahead of time, Sparse AutoEncoders (SAEs) learn a dictionary of concept-like features directly from a model’s internal activations [15], and have become a central tool of Mechanistic Interpretability (MechInt) — the effort to reverse-engineer the computations

¹ The width of the unsupervised layer is fixed at 768 in the authors’ released code (<https://github.com/Trustworthy-ML-Lab/CB-LLMs>); the paper itself leaves it unspecified.

of neural networks, and of Large Language Models in particular. This unsupervised discovery of interpretable features is the line of work on which this thesis builds, and it is treated in full in section 2.3.3.

2.3 AUTOENCODERS

An autoencoder is a neural network trained to reconstruct its input through an intermediate latent representation [12, Ch 14]. It consists of two components: an **encoder** $f_\phi : \mathbb{R}^n \rightarrow \mathbb{R}^m$ that maps an input $\mathbf{x}$ to a latent code $\mathbf{z} = f_\phi(\mathbf{x})$, and a **decoder** $g_\theta : \mathbb{R}^m \rightarrow \mathbb{R}^n$ that reconstructs the input as $\hat{\mathbf{x}} = g_\theta(\mathbf{z})$. Training minimises a reconstruction loss, typically the Mean Squared Error (MSE):

$$\mathcal{L}_{\text{recon}} = \text{MSE}(\mathbf{x}, \hat{\mathbf{x}}) = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2. \quad (2.1)$$

When $m < n$, the bottleneck forces the model to learn a compressed representation. When $m > n$, the network is overcomplete and additional constraints are required to prevent the trivial solution of learning the identity. In either case, the structure of the latent space is shaped by the choice of architecture and regularisation.

2.3.1 STANDARD AUTOENCODERS

2.3.1.1 Applications

IMAGE RETRIEVAL Krizhevsky and Hinton [22] proposed using deep autoencoders for content-based image retrieval. Their key insight is that a deep autoencoder can learn to map images to short binary codes that preserve semantic similarity: images that are perceptually similar are mapped to nearby codes in the latent space, allowing retrieval to be reduced to a fast lookup rather than an exhaustive search over the full database. They demonstrate this on a dataset of 1.6 million tiny images, showing that codes produced by deep autoencoders yield qualitatively and quantitatively better retrieval results than matching in raw pixel space. This work illustrated that the compressed latent representation learned by an autoencoder need not merely serve reconstruction, but can itself be a semantically meaningful and practically useful object.

ANOMALY DETECTION Sakurada and Yairi [35] were among the first to explicitly propose autoencoders as a tool for anomaly detection. Their core insight is that an autoencoder trained on normal data learns a compressed representation of the normal data distribution; at test time, inputs that deviate from this distribution are reconstructed poorly, and the resulting reconstruction error serves as an anomaly score. To substantiate this claim, they apply autoencoders to both synthetic data generated from the Lorenz system and real spacecraft

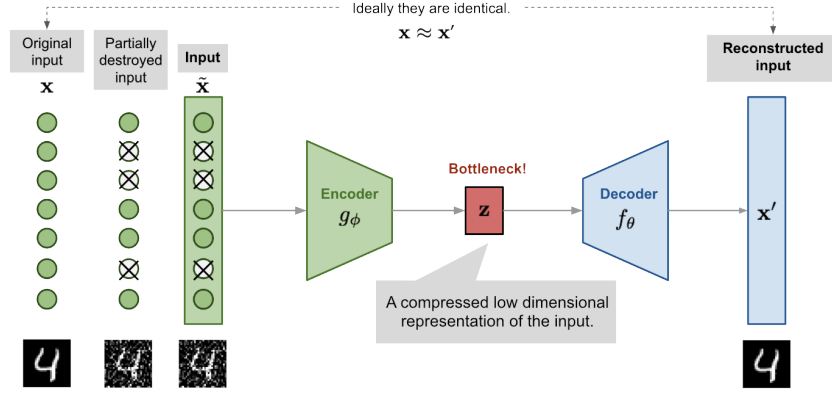


Figure 2.1: Illustration of Denoising autoencoder model architecture, reproduced from Weng [46]. Note: this figure uses f for the decoder and g for the encoder, the reverse of the letter convention adopted in this thesis.

telemetry, comparing against linear Principal Component Analysis (PCA) and kernel PCA. They demonstrate that autoencoders, by virtue of their nonlinear dimensionality reduction, are able to detect subtle anomalies that linear PCA misses, while remaining computationally lighter than kernel PCA. This work established reconstruction error as a natural unsupervised anomaly signal for autoencoder-based methods.

2.3.1.2 Denoising AutoEncoders

The denoising autoencoder was introduced by Vincent et al. [44] to avoid overfitting and improve the robustness of the learned representation. The input is partially corrupted by adding noise to or masking some values of the input vector in a stochastic manner, $\mathbf{x}_c \sim \mathcal{M}_{\mathcal{D}}(\mathbf{x}_c | \mathbf{x})$, as shown in fig. 2.1. The model is then trained to reconstruct the original uncorrupted input:

$$\mathcal{L}_{\text{DAE}} = \|\mathbf{x} - g_\theta(f_\phi(\mathbf{x}_c))\|_2^2. \quad (2.2)$$

This design is motivated by the fact that humans can easily recognize an object or a scene even the view is partially occluded or corrupted. To “repair” the partially destroyed input, the denoising autoencoder has to discover and capture relationship between dimensions of input in order to infer missing pieces.

For high dimensional input with high redundancy, like images, the model is likely to depend on evidence gathered from a combination of many input dimensions to recover the denoised version rather than to overfit one dimension. This builds up a good foundation for learning *robust* latent representation.

The noise is controlled by a stochastic mapping $\mathcal{M}_{\mathcal{D}}(\mathbf{x}_c | \mathbf{x})$, and it is not specific to a particular type of corruption process (i. e., masking noise, Gaussian noise, salt-and-pepper noise, etc.). Naturally the corruption process can be equipped with prior knowledge [46].

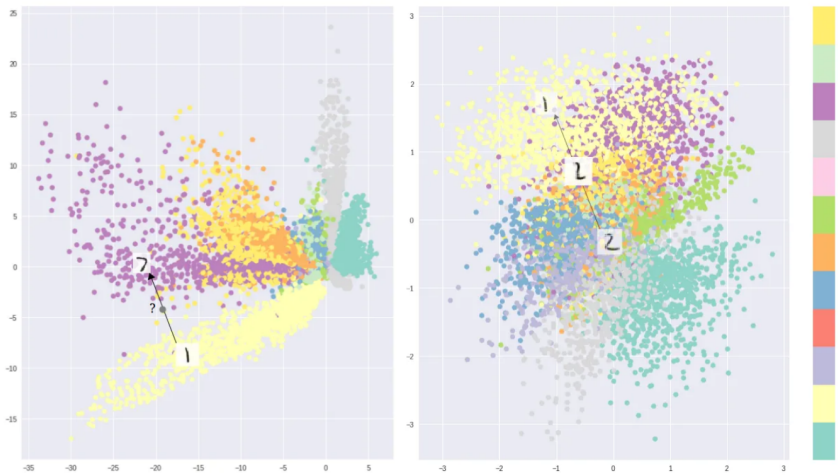


Figure 2.2: Two-dimensional latent spaces learned on MNIST, coloured by digit class, reproduced from Shafkat [37]. Optimising for reconstruction alone (left) lets a standard autoencoder separate the classes into disconnected clusters with gaps that decode to unrealistic samples, whereas the Variational AutoEncoder objective (right) yields a smoother, more continuous arrangement, concentrated around the origin by the prior, that can be sampled and interpolated.

2.3.2 VARIATIONAL AUTOENCODERS

Variational AutoEncoders (VAEs) [19, 31] reinterpret the autoencoder as a probabilistic generative model. Rather than mapping an input to a single latent point, the encoder maps it to a *distribution* over latent codes, and the decoder is treated as a generative model that defines a distribution over inputs given a latent code. This probabilistic framing turns the autoencoder into a model from which new data can be sampled, and forces its latent space to have a smooth, continuous structure.

This smoothness is the property that sets a VAE apart from a standard autoencoder. Trained for reconstruction alone, an autoencoder is free to scatter its encodings into well-separated clusters with large gaps between them; those gaps are regions that decode to unrealistic outputs, so the space cannot be sampled or interpolated reliably. The VAE objective instead pushes encodings to fill the latent space more uniformly and to vary smoothly, so that nearby codes decode to similar inputs and paths between codes stay on the data manifold. Figure 2.2 illustrates the difference on a two-dimensional latent space trained on MNIST.

2.3.2.1 Variational Inference

Formally, a VAE models the data through a generative process in which a latent code is first drawn from a prior $\mathbf{z} \sim p(\mathbf{z})$ — typically

a standard Gaussian $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ — and an observation is then drawn from a decoder distribution $p_\theta(\mathbf{x} | \mathbf{z})$ parameterised by a neural network. Training would ideally maximise the marginal likelihood

$$p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x} | \mathbf{z}) p(\mathbf{z}) d\mathbf{z},$$

but this integral, and the corresponding posterior $p_\theta(\mathbf{z} | \mathbf{x}) = p_\theta(\mathbf{x} | \mathbf{z})p_\theta(\mathbf{z})/p_\theta(\mathbf{x})$, are intractable for any non-trivial decoder.

The key idea of Kingma and Welling [19] is to sidestep this intractability with *amortised variational inference*: an encoder network $q_\phi(\mathbf{z} | \mathbf{x})$ is trained to approximate the true posterior, with all data points sharing the same inference parameters ϕ . The encoder outputs the parameters of a diagonal Gaussian,

$$q_\phi(\mathbf{z} | \mathbf{x}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}_\phi(\mathbf{x}), \text{diag}(\boldsymbol{\sigma}_\phi^2(\mathbf{x}))), \quad (2.3)$$

so that each input is mapped to a mean and a (diagonal) covariance rather than to a single point.

Instead of the intractable likelihood, the VAE maximises a tractable lower bound on it, the Evidence Lower Bound (ELBO):

$$\log p_\theta(\mathbf{x}) \geq \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x} | \mathbf{z})]}_{\text{reconstruction}} - \underbrace{D_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{x}) \| p(\mathbf{z}))}_{\text{regularisation}}. \quad (2.4)$$

The bound follows from the identity

$$\log p_\theta(\mathbf{x}) = \text{ELBO} + D_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{x}) \| p_\theta(\mathbf{z} | \mathbf{x})),$$

together with the non-negativity of the Kullback–Leibler (KL) divergence; it is tight precisely when the approximate posterior matches the true posterior $p_\theta(\mathbf{z} | \mathbf{x})$. Maximising the ELBO simultaneously pushes the decoder to assign high likelihood to the data (the reconstruction term) and keeps the approximate posterior close to the prior (the KL divergence term). Negating it yields the training loss:

$$\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{recon}} + D_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{x}) \| p(\mathbf{z})). \quad (2.5)$$

When the decoder is Gaussian with fixed isotropic variance, the first term reduces, up to an additive constant, to the MSE reconstruction loss of eq. (2.1); the second term, being the divergence between two Gaussians, has a closed form. For the standard-normal prior it becomes:

$$D_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{x}) \| p(\mathbf{z})) = \frac{1}{2} \sum_{j=1}^m (\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1),$$

where μ_j and σ_j are the components of the encoder’s predicted mean and standard deviation.

Evaluating the reconstruction term requires sampling $\mathbf{z}$ from $q_\phi(\mathbf{z} | \mathbf{x})$, an operation that is not differentiable with respect to

the encoder parameters and would therefore block gradient-based training. The VAE resolves this with the *reparameterisation trick*: the stochasticity is moved to an auxiliary noise variable, and the sample is expressed as a deterministic, differentiable function of the encoder outputs,

$$\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}),$$

where $\odot$ denotes the element-wise product. Gradients can now flow through $\boldsymbol{\mu}_\phi$ and $\boldsymbol{\sigma}_\phi$ while the randomness is confined to $\boldsymbol{\epsilon}$, making the whole objective trainable end-to-end by stochastic gradient descent.

2.3.2.2 Architectures

There exist several variants that modify the base VAE to shape the latent space or the form of the latent code; two are particularly relevant here.

β -VAE Higgins et al. [14] introduced a single modification: a scalar weight β on the KL term of the objective,

$$\mathcal{L}_{\beta\text{-VAE}} = \mathcal{L}_{\text{recon}} + \beta D_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{x}) \| p(\mathbf{z})). \quad (2.6)$$

Setting $\beta > 1$ places additional pressure on the approximate posterior to match the factorised prior, which encourages individual latent dimensions to capture statistically independent factors of variation — a property known as *disentanglement*. The standard VAE is recovered at $\beta = 1$. Changing β offers a trade-off: a larger value improves the structure and interpretability of the latent space at the cost of reconstruction fidelity.

vQ-VAE van den Oord, Vinyals and Kavukcuoglu [43] replaced the continuous Gaussian latent with a *discrete* one. The encoder output is quantised to the nearest entry of a learned codebook $\{\mathbf{e}_1, \dots, \mathbf{e}_K\}$, so that each latent is represented by a discrete index rather than a continuous vector. This sidesteps the *posterior collapse* that can affect continuous VAEs paired with expressive autoregressive decoders, in which the decoder learns to ignore the latent entirely. Because the nearest-neighbour lookup is non-differentiable, gradients are passed to the encoder with a straight-through estimator and the codebook is learned through dedicated loss terms. Vector-Quantized VAE underpins many modern generative systems, providing the discrete latent vocabulary over which an autoregressive prior is subsequently trained.

2.3.2.3 Applications

VAEs are used both as practical components in large generative systems and for interpretable representation learning; the latter is the more relevant to this thesis.

GENERATIVE MODELLING Although a VAE can act as a standalone generator, in modern systems it more often serves as a *compression backbone* for other generative models: latent diffusion models run the diffusion process in the compact latent space of an autoencoder rather than in pixel space. The autoencoder behind Stable Diffusion [33] is regularised only weakly, with a small KL penalty (in the manner of a VAE) or with vector quantisation. Discrete VQ-VAEs [43] encode data into a grid of discrete codes from a learned codebook, which an autoregressive prior can then generate. The smooth latent also serves as a search space: optimising over it has been used to search for new molecules with desired properties [11].

ANOMALY DETECTION Like standard autoencoders, VAEs can flag anomalies, but they replace the single reconstruction error with a probabilistic score: the reconstruction probability, or the ELBO, gives a principled measure of how well the model explains an input [3]. This is the probabilistic counterpart of the approach described in section 2.3.1.1.

REPRESENTATION LEARNING VAEs are also a standard tool for unsupervised representation learning. The β -VAE (section 2.3.2.2) is the canonical example: up-weighting the KL term pressures the model towards *disentangled* latents whose individual dimensions capture independent, semantically meaningful factors of variation [14], yielding structured low-dimensional features for downstream use.

INTERPRETABILITY The structured latent that makes VAEs good representation learners also makes them attractive for explainability. Their smoothness supports *counterfactual explanation* (section 2.2.1.2), as perturbing the latent yields plausible, on-manifold variants of an input [16]. A substantial line of work uses VAEs for *causal representation learning*, aligning latent factors with underlying causal variables [13, 21]. VAEs have also been paired with concept-based models such as self-explaining networks [1]. Closest to this thesis, variational ideas have been brought to sparse dictionary learning — variational sparse coding [42] and variational Sparse AutoEncoders [4] — seeking to combine the interpretability of sparse codes with the structured latent of a VAE, the same combination developed in section 3.3.

2.3.3 SPARSE AUTOENCODERS

Sparse AutoEncoders (SAEs) are one of the main variants of autoencoders. Just like autoencoders, they learn a dictionary of feature directions from the input data, but they typically are overcomplete ($m \gg n$)

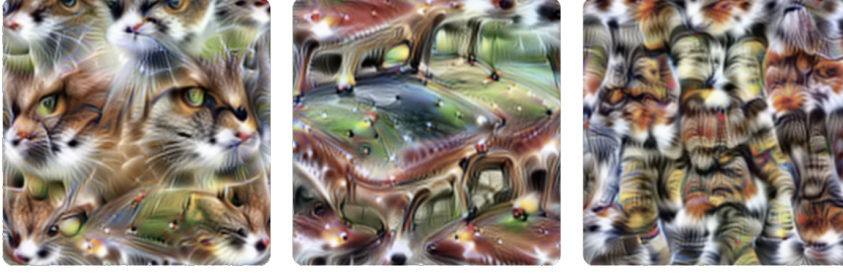


Figure 2.3: Neuron 4e:55, a polysemantic neuron from InceptionV1 which responds to cat faces, fronts of cars, and cat legs. Reproduced from Olah et al. [27].

and their latent representation is forced to be sparse using some kind of sparsity mechanism. Given input $\mathbf{x} \in \mathbb{R}^n$, the [SAE](#) encodes it as:

$$\mathbf{z} = \text{ReLU}(\mathbf{W}_e(\mathbf{x} - \mathbf{b}_{\text{pre}}) + \mathbf{b}_e), \quad (2.7)$$

where $\mathbf{W}_e \in \mathbb{R}^{m \times n}$ is the encoder weight matrix, $\mathbf{b}_e \in \mathbb{R}^m$ is the encoder bias, and $\mathbf{b}_{\text{pre}} \in \mathbb{R}^n$ is a pre-encoder bias subtracted from the input before encoding. This learned offset allows the encoder to operate on roughly mean-centered activations, improving training stability [6]. The decoder then reconstructs the input as:

$$\hat{\mathbf{x}} = \mathbf{W}_d \mathbf{z} + \mathbf{b}_{\text{pre}}, \quad (2.8)$$

where $\mathbf{W}_d \in \mathbb{R}^{n \times m}$ is the decoder weight matrix. Each column $\mathbf{d}_i$ of $\mathbf{W}_d$ is a dictionary atom, representing a candidate feature direction in activation space.

When used in the context of [MechInt](#), the overcompleteness and the sparsity constraint are motivated by the *superposition hypothesis*.

2.3.3.1 The Superposition Hypothesis

A central challenge in [MechInt](#) is that individual neurons in neural networks tend to be *polysemantic*: they activate in response to multiple, seemingly unrelated concepts [8, 27]. An example of this can be seen in fig. 2.3. This runs contrary to the intuition that an interpretable model should have neurons with clean, well-defined roles.

Elhage et al. [8] proposed the *superposition hypothesis* to explain this phenomenon. The core idea is that a network may represent more features than it has dimensions by exploiting the fact that natural inputs are sparse: at any given time, only a small fraction of all possible features are relevant. Under this condition, a network can pack many near-orthogonal feature directions into a lower-dimensional space with limited interference; polysemanticity ceases to be a failure of the model, and instead becomes a rational compression strategy under capacity constraints.

This has a direct implication for interpretability: the natural basis of a model’s activations is not the neuron basis, but a larger, overcomplete set of feature directions embedded within it. Recovering this basis requires moving beyond individual neuron analysis, towards methods that can identify sparse, distributed structure. This is precisely the goal of *sparse dictionary learning* [28], a classical technique from computational neuroscience that SAEs adapt to the setting of modern language models.

2.3.3.2 Architectures

There are many ways to enforce sparsity, each with its pros and cons. Here is a selection of methods that range from the initial SAE idea to more modern architectures that try to improve on it.

VANILLA ℓ_1 This is the original architecture proposed by Huben et al. [15]. It uses an ℓ_1 penalty² on the latent code to encourage most of $\mathbf{z}$ ’s entries to be exactly zero, so that any given activation is reconstructed from a small number of active features. The loss used to train this model is the following:

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \lambda \|\mathbf{z}\|_1. \quad (2.9)$$

The scalar λ controls the sparsity–reconstruction trade-off and must be tuned carefully: too large, and reconstruction degrades; too small, and the learned features lose monosemanticity. A known side effect of ℓ_1 regularisation is *shrinkage* [41, 47]: beyond driving inactive features to zero, the penalty also biases active feature magnitudes below their reconstruction-optimal values, causing the model to systematically underestimate the true activation of each feature [29, 47]. Constraining the decoder columns to unit ℓ_2 norm after each gradient step partially mitigates this, by preventing the model from reducing the ℓ_1 penalty through the trivial reparameterisation of scaling down encoder outputs and compensating with larger decoder norms [29]. There is, however, a residual gradient-level bias that is structurally entangled with the ℓ_1 penalty and cannot be resolved with this trick.

TOPK SAEs Gao et al. [10] proposed replacing the ℓ_1 penalty with a hard sparsity constraint, implemented via a TopK activation function that retains only the k largest pre-activations and zeros the rest. This directly controls the number of active features per forward pass, eliminating the need to tune λ and removing the shrinkage bias entirely. The full training objective is:

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \alpha \mathcal{L}_{\text{aux}}, \quad (2.10)$$

² The ℓ_p norm is defined as $\ell_p(\mathbf{x}) = \|\mathbf{x}\|_p = (\sum_i |x_i|^p)^{1/p}$, $p \geq 1$.

where the auxiliary loss $\mathcal{L}_{\text{aux}}$ addresses the problem of dead latents, i. e., features that cease to activate entirely during training. Latents are flagged as dead if they have not activated for a predetermined number of tokens. Given the reconstruction residual $\mathbf{e} = \mathbf{x} - \hat{\mathbf{x}}$, the top- k_{aux} dead latents are selected and used to reconstruct $\mathbf{e}$; the resulting reconstruction error is $\mathcal{L}_{\text{aux}}$. This gives dead latents a gradient signal proportional to how much of the residual they could explain, encouraging them to activate rather than remain permanently dormant. Gao et al. [10] demonstrate that TopK SAEs achieve better reconstruction at equivalent sparsity levels compared to ℓ_1 baselines, making them a strong and widely used alternative.

GATED SAES Rajamanoharan et al. [29] introduced an architecture that separates the decision of whether a feature is active from the decision of how strongly it activates. A dedicated gating pathway produces a binary mask via a Heaviside step function (approximated during training with a straight-through estimator) which is applied element-wise to the encoder output. This architectural separation allows the model to learn feature presence and magnitude more independently, addressing shrinkage from a structural rather than a loss-based perspective.

JUMPRELU SAES Rajamanoharan et al. [30] proposed replacing the standard ReLU with a JumpReLU activation, which is zero below a learned per-feature threshold θ_i and equal to the pre-activation above it:

$$\text{JumpReLU}_{\theta_i}(z_i) = z_i \cdot \mathbb{1}[z_i > \theta_i]. \quad (2.11)$$

The key advantage of this parameterisation is that it provides a dedicated, per-feature learnable threshold, which allows the ℓ_0 norm³ of the feature activations — $\sum_i H(\pi_i - \theta_i)$, where π_i are the encoder pre-activations and $H(x)$ is the Heaviside step function — to be trained directly via a straight-through estimator applied to the threshold parameter θ_i . Because each threshold can be positioned near the actual boundary between active and inactive pre-activations for its feature, the gradient signal is stable and controllable, making ℓ_0 training practical in a way that would not be possible with a fixed threshold. Training with ℓ_0 rather than ℓ_1 eliminates shrinkage entirely, since ℓ_0 penalises only the number of active features and not their intensity. JumpReLU SAEs achieve comparable performance to TopK SAEs.

2.3.3.3 Applications

Unlike general-purpose autoencoders and VAEs, which are applied across a range of domains, SAEs find their primary application in

³ The ℓ_0 norm is a pseudo-norm defined as the number of non-zero vector components: $\ell_0(\mathbf{x}) = \sum_i \mathbb{1}[x_i > 0]$.

Make sure to mention what the residual stream, Multi-Layer Perceptron (MLP), and attention heads are in a chapter about LLMs

interpreting the internal activations of neural networks, and [MechInt](#) remains their most prominent application, making it the focus of this section. [SAEs](#) can be trained on activations from any layer or component of a transformer; common targets include the residual stream at various depths, [MLP](#) layer outputs, and attention head outputs [6, 15]. The resulting dictionary then supports the applications described below.

FEATURE DISCOVERY The primary use of [SAEs](#) is *feature discovery*: training a [SAE](#) on a model’s internal activations and examining the resulting dictionary atoms to identify human-interpretable concepts. The underlying hypothesis is that each atom, active for only a semantically coherent set of inputs, corresponds to a monosemantic feature of the model’s computation — a clean unit of meaning that the model had obscured in the neuron basis. In practice these features often correspond to recognisable concepts: Bricken et al. [6] trained a [SAE](#) on the [MLP](#) of a one-layer transformer and found features responsive to specific tokens, syntactic roles, and semantic categories, while at production scale Templeton et al. [40] report features for famous people, countries, emotions, or even biases (like racism, sexism, hatred) and sarcastic praise. These results provide strong empirical support for the superposition hypothesis and suggest that [SAEs](#) can serve as a practical lens through which to study what information [LLMs](#) represent.

STEERING The features recovered by a [SAE](#) are not merely descriptive: because each corresponds to a direction in activation space, its activation can be clamped to an arbitrary value during a forward pass to causally *steer* the model’s behaviour. Templeton et al. [40] demonstrated this by amplifying a single feature, causing the model to compulsively reference the Golden Gate Bridge regardless of the input. Such interventions show that the discovered features are causally implicated in the model’s computation rather than merely correlated with its inputs, and open a path to controllable generation and targeted safety interventions.

CIRCUIT ANALYSIS Beyond cataloguing individual features, [SAEs](#) can serve as the building blocks for reconstructing the *circuits* a model uses to perform a computation, expressing the interactions between features across layers as an interpretable causal graph [24]. More recent work favours *transcoders* — a variant trained to approximate the input–output behaviour of an [MLP](#) sublayer with a sparse hidden representation, rather than to reconstruct a single activation — which should disentangle the model’s computation more cleanly and yield sparser, more faithful circuits [2, 7].

2.3.3.4 Evaluation

Assessing the quality of learned features is non-trivial, as interpretability is inherently subjective. Several proxy metrics are used in practice. *Reconstruction quality* is judged by its effect on the model rather than by the raw reconstruction error: the SAE’s output is substituted for the original activations, the model is run forward, and the resulting increase in its next-token Cross-Entropy loss is measured [10]. A faithful SAE leaves this loss almost unchanged, indicating that it has retained the information the model actually uses. *Sparsity* is quantified via the average ℓ_0 norm of the latent codes, i. e., the mean number of active features per token. *Automated interpretability* pipelines use a second language model to generate and score natural-language descriptions of individual features [5], providing a scalable if imperfect proxy for human judgement.

CONCLUSIONS

METHODS

3.1 DEFINITIONS/NOTATION

3.2 AIM OF THE WORK

3.3 MODEL DEVELOPMENT

3.3.1 PROBABILISTIC FORMULATION

3.3.2 ARCHITECTURES

CONCLUSIONS

EXPERIMENTS

4.1 MODEL TRAINING

4.2 EVALUATION METRICS

4.2.1 MSE

4.2.2 ℓ_0

4.2.3 PERPLEXITY

4.3 RESULTS

CONCLUSIONS

CONCLUSIONS

5.1 CONTRIBUTIONS

5.2 FUTURE WORK

BIBLIOGRAPHY

- [1] David Alvarez Melis and Tommi Jaakkola. ‘Towards Robust Interpretability with Self-Explaining Neural Networks’. In: *Advances in Neural Information Processing Systems*. Vol. 31. Curran Associates, Inc., 2018. URL: https://papers.nips.cc/paper_files/paper/2018/hash/3e9f0fc9b2f89e043bc6233994dfcf76-Abstract.html (visited on 11/06/2026).
- [2] Emmanuel Ameisen et al. *Circuit Tracing: Revealing Computational Graphs in Language Models*. Tech. rep. Transformer Circuits Thread, Mar. 2025. URL: <https://transformer-circuits.pub/2025/attribution-graphs/methods.html> (visited on 11/06/2026).
- [3] Jinwon An and Sungzoon Cho. *Variational Autoencoder Based Anomaly Detection Using Reconstruction Probability*. Dec. 2015. URL: <http://dm.snu.ac.kr/static/docs/TR/SNUDM-TR-2015-03.pdf>.
- [4] Zachary Baker and Yuxiao Li. *Analysis of Variational Sparse Autoencoders*. Comment: 15 pages, 11 figures. Oct. 2025. DOI: [10.48550/arXiv.2509.22994](https://doi.org/10.48550/arXiv.2509.22994). arXiv: [2509.22994](https://arxiv.org/abs/2509.22994) [cs]. URL: <http://arxiv.org/abs/2509.22994> (visited on 20/10/2025).
- [5] Steven Bills, Nick Cammarata, Dan Mossing, Henk Tillman, Leo Gao, Gabriel Goh, Ilya Sutskever, Jan Leike, Jeff Wu and William Saunders. *Language Models Can Explain Neurons in Language Models*. Tech. rep. OpenAI, May 2023. URL: <https://openaipublic.blob.core.windows.net/neuron-explainer/paper/index.html> (visited on 11/06/2026).
- [6] Trenton Bricken et al. *Towards Monosemanticity: Decomposing Language Models With Dictionary Learning*. Tech. rep. Transformer Circuits Thread, Oct. 2023. URL: <https://transformer-circuits.pub/2023/monosemantic-features/index.html> (visited on 11/06/2026).
- [7] Jacob Dunefsky, Philippe Chlenski and Neel Nanda. ‘Transcoders Find Interpretable LLM Feature Circuits’. In: *Advances in Neural Information Processing Systems* 37 (Dec. 2024), pp. 24375–24410. DOI: [10.52202/079017-0768](https://doi.org/10.52202/079017-0768). URL: https://proceedings.neurips.cc/paper_files/paper/2024/hash/2b8f4db0464cc5b6e9d5e6bea4b9f308-Abstract-Conference.html (visited on 10/06/2026).

- [8] Nelson Elhage et al. *Toy Models of Superposition*. Tech. rep. Transformer Circuits Thread, Sept. 2022. URL: https://transformer-circuits.pub/2022/toy_model/index.html (visited on 11/06/2026).
- [9] Mateo Espinosa Zarlenga et al. 'Concept Embedding Models: Beyond the Accuracy-Explainability Trade-Off'. In: *Advances in Neural Information Processing Systems* 35 (Dec. 2022), pp. 21400–21413. URL: https://proceedings.neurips.cc/paper_files/paper/2022/hash/867c06823281e506e8059f5c13a57f75-Abstract-Conference.html (visited on 12/06/2026).
- [10] Leo Gao, Tom Dupre la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike and Jeffrey Wu. 'Scaling and Evaluating Sparse Autoencoders'. In: *The Thirteenth International Conference on Learning Representations*. Oct. 2024. URL: <https://openreview.net/forum?id=tcsZt9ZNKD> (visited on 14/05/2026).
- [11] Rafael Gómez-Bombarelli, Jennifer N. Wei, David Duvenaud, José Miguel Hernández-Lobato, Benjamín Sánchez-Lengeling, Dennis Sheberla, Jorge Aguilera-Iparraguirre, Timothy D. Hirzel, Ryan P. Adams and Alán Aspuru-Guzik. 'Automatic Chemical Design Using a Data-Driven Continuous Representation of Molecules'. In: *ACS Central Science* 4.2 (Feb. 2018), pp. 268–276. ISSN: 2374-7943. DOI: [10.1021/acscentsci.7b00572](https://doi.org/10.1021/acscentsci.7b00572). URL: <https://doi.org/10.1021/acscentsci.7b00572> (visited on 10/06/2026).
- [12] Ian Goodfellow, Aaron Courville and Yoshua Bengio. *Deep Learning*. Adaptive Computation and Machine Learning. "Deep learning is a form of machine learning that enables computers to learn from experience and understand the world in terms of a hierarchy of concepts. Because the computer gathers knowledge from experience, there is no need for a human computer operator to formally specify all the knowledge that the computer needs. The hierarchy of concepts allows the computer to learn complicated concepts by building them out of simpler ones; a graph of these hierarchies would be many layers deep. This book introduces a broad range of topics in deep learning. The text offers mathematical and conceptual background, covering relevant concepts in linear algebra, probability theory and information theory, numerical computation, and machine learning. It describes deep learning techniques used by practitioners in industry, including deep feedforward networks, regularization, optimization algorithms, convolutional networks, sequence modeling, and practical methodology; and it surveys such applications as natural language processing, speech recognition, computer vision, online recommendation systems, bioinformat-

ics, and video games. Finally, the book offers research perspectives, covering such theoretical topics as linear factor models, autoencoders, representation learning, structured probabilistic models, Monte Carlo methods, the partition function, approximate inference, and deep generative models. Deep Learning can be used by undergraduate or graduate students planning careers in either industry or research, and by software engineers who want to begin using deep learning in their products or platforms. A website offers supplementary material for both readers and instructors"—Publisher's description Introduction – Applied math and machine learning basics. Linear algebra – Probability and information theory – Numerical computation – Machine learning basics – Deep networks: modern practices. Deep feedforward networks – Regularization for deep learning – Optimization for training deep models – Convolutional networks – Sequence modeling: recurrent and recursive nets – Practical methodology – Applications – Deep learning research. Linear factor models – Autoencoders – Representation learning – Structured probabilistic models for deep learning – Monte Carlo methods – Confronting the partition function – Approximate inference – Deep generative models
Includes bibliographical references and index. Cambridge, Massachusetts: The MIT Press, 2016. ISBN: 978-0-262-03561-3 978-0-262-33737-3.

- [13] Yash Goyal, Amir Feder, Uri Shalit and Been Kim. *Explaining Classifiers with Causal Concept Effect (CaCE)*. Feb. 2020. DOI: [10.48550/arXiv.1907.07165](https://doi.org/10.48550/arXiv.1907.07165). arXiv: [1907.07165 \[cs.LG\]](https://arxiv.org/abs/1907.07165). URL: <http://arxiv.org/abs/1907.07165> (visited on 01/06/2026).
- [14] Irina Higgins, Loic Matthey, Arka Pal, Christopher Burgess, Xavier Glorot, Matthew Botvinick, Shakir Mohamed and Alexander Lerchner. 'Beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework'. In: *International Conference on Learning Representations*. Feb. 2017. URL: <https://openreview.net/forum?id=Sy2fzU9gl> (visited on 10/06/2026).
- [15] Robert Huben, Hoagy Cunningham, Logan Riggs Smith, Aidan Ewart and Lee Sharkey. 'Sparse Autoencoders Find Highly Interpretable Features in Language Models'. In: *The Twelfth International Conference on Learning Representations*. Oct. 2023. URL: <https://openreview.net/forum?id=F76bwRSLeK> (visited on 28/05/2026).
- [16] Shalmali Joshi, Oluwasanmi Koyejo, Warut Vijitbenjaronk, Been Kim and Joydeep Ghosh. *Towards Realistic Individual Recourse and Actionable Explanations in Black-Box Decision Making Systems*. July 2019. DOI: [10.48550/arXiv.1907.09615](https://doi.org/10.48550/arXiv.1907.09615). arXiv: [1907.09615](https://arxiv.org/abs/1907.09615)

- [cs.LG]. URL: <http://arxiv.org/abs/1907.09615> (visited on 11/06/2026).
- [17] Been Kim, Rajiv Khanna and Oluwasanmi O Koyejo. ‘Examples Are Not Enough, Learn to Criticize! Criticism for Interpretability’. In: *Advances in Neural Information Processing Systems*. Vol. 29. Curran Associates, Inc., 2016. URL: https://proceedings.neurips.cc/paper_files/paper/2016/hash/5680522b8e2bb01943234bce7bf84534-Abstract.html (visited on 11/06/2026).
- [18] Been Kim, Martin Wattenberg, Justin Gilmer, Carrie Cai, James Wexler, Fernanda Viegas and Rory Sayres. ‘Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV)’. In: *Proceedings of the 35th International Conference on Machine Learning*. PMLR, July 2018, pp. 2668–2677. URL: <https://proceedings.mlr.press/v80/kim18d.html> (visited on 11/06/2026).
- [19] Diederik P. Kingma and Max Welling. ‘Auto-Encoding Variational Bayes’. In: Useful for the reparametrization trick (sec 2.4). International Conference on Learning Representations, May 2014. DOI: [10.48550/arXiv.1312.6114](https://doi.org/10.48550/arXiv.1312.6114). arXiv: [1312.6114](https://arxiv.org/abs/1312.6114) [stat]. URL: <http://arxiv.org/abs/1312.6114> (visited on 07/11/2025).
- [20] Pang Wei Koh, Thao Nguyen, Yew Siang Tang, Stephen Mussmann, Emma Pierson, Been Kim and Percy Liang. ‘Concept Bottleneck Models’. In: *Proceedings of the 37th International Conference on Machine Learning*. PMLR, Nov. 2020, pp. 5338–5348. URL: <https://proceedings.mlr.press/v119/koh20a.html> (visited on 11/06/2026).
- [21] Aneesh Komanduri, Yongkai Wu, Wen Huang, Feng Chen and Xintao Wu. ‘SCM-VAE: Learning Identifiable Causal Representations via Structural Knowledge’. In: *2022 IEEE International Conference on Big Data (Big Data)*. Dec. 2022, pp. 1014–1023. DOI: [10.1109/BigData55660.2022.10021114](https://doi.org/10.1109/BigData55660.2022.10021114). URL: <https://ieeexplore.ieee.org/document/10021114> (visited on 21/04/2026).
- [22] Alex Krizhevsky and Geoffrey E. Hinton. ‘Using Very Deep Autoencoders for Content-Based Image Retrieval’. In: *19th European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning* (Apr. 2011).
- [23] Scott M Lundberg and Su-In Lee. ‘A Unified Approach to Interpreting Model Predictions’. In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., 2017. URL: https://proceedings.neurips.cc/paper_files/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html (visited on 01/06/2026).

- [24] Samuel Marks, Can Rager, Eric Michaud, Yonatan Belinkov, David Bau and Aaron Mueller. ‘Sparse Feature Circuits: Discovering and Editing Interpretable Causal Graphs in Language Models’. In: *International Conference on Learning Representations* 2025 (May 2025), pp. 23888–23923. URL: https://proceedings.iclr.cc/paper_files/paper/2025/hash/3ba4d47a83e498c2b1a0868cba20f6de-Abstract-Conference.html (visited on 10/06/2026).
- [25] Christoph Molnar. *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. 3rd edition. Munich, Germany: Christoph Molnar, 2025. ISBN: 978-3-911578-03-5.
- [26] Tuomas Oikarinen, Subhro Das, Lam M. Nguyen and Tsui-Wei Weng. ‘Label-Free Concept Bottleneck Models’. In: *The Eleventh International Conference on Learning Representations*. Sept. 2022. URL: <https://openreview.net/forum?id=FlCg47MNvBA> (visited on 11/06/2026).
- [27] Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov and Shan Carter. ‘Zoom In: An Introduction to Circuits’. In: *Distill* 5.3 (Mar. 2020), e00024.001. ISSN: 2476-0757. DOI: [10.23915/distill.00024.001](https://doi.org/10.23915/distill.00024.001). URL: <https://distill.pub/2020/circuits/zoom-in> (visited on 04/05/2026).
- [28] Bruno A. Olshausen and David J. Field. ‘Sparse Coding with an Overcomplete Basis Set: A Strategy Employed by V1?’ In: *Vision Research* 37.23 (Dec. 1997), pp. 3311–3325. ISSN: 0042-6989. DOI: [10.1016/S0042-6989\(97\)00169-7](https://doi.org/10.1016/S0042-6989(97)00169-7). URL: <https://www.sciencedirect.com/science/article/pii/S0042698997001697> (visited on 28/05/2026).
- [29] Senthooran Rajamanoharan, Arthur Conmy, Lewis Smith, Tom Lieberum, Vikrant Varma, János Kramár, Rohin Shah and Neel Nanda. ‘Improving Sparse Decomposition of Language Model Activations with Gated Sparse Autoencoders’. In: *Advances in Neural Information Processing Systems* 37 (Dec. 2024), pp. 775–818. DOI: [10.52202/079017-0024](https://doi.org/10.52202/079017-0024). URL: https://proceedings.neurips.cc/paper_files/paper/2024/hash/01772a8b0420baec00c4d59fe2fbace6-Abstract-Conference.html (visited on 05/06/2026).
- [30] Senthooran Rajamanoharan, Tom Lieberum, Nicolas Sonnerat, Arthur Conmy, Vikrant Varma, János Kramár and Neel Nanda. *Jumping Ahead: Improving Reconstruction Fidelity with JumpReLU Sparse Autoencoders*. Comment: v2: new appendix H comparing kernel functions & bug-fixes to pseudo-code in Appendix J v3: further bug-fix to pseudo-code in Appendix J. Aug. 2024. DOI: [10.48550/arXiv.2407.14435](https://doi.org/10.48550/arXiv.2407.14435). arXiv: [2407.14435 \[cs\]](https://arxiv.org/abs/2407.14435). URL: <http://arxiv.org/abs/2407.14435> (visited on 23/02/2026).

- [31] Danilo Jimenez Rezende, Shakir Mohamed and Daan Wierstra. ‘Stochastic Backpropagation and Approximate Inference in Deep Generative Models’. In: *Proceedings of the 31st International Conference on Machine Learning*. PMLR, June 2014, pp. 1278–1286. URL: <https://proceedings.mlr.press/v32/rezende14.html> (visited on 10/06/2026).
- [32] Marco Tulio Ribeiro, Sameer Singh and Carlos Guestrin. “‘Why Should I Trust You?’: Explaining the Predictions of Any Classifier”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD ’16. New York, NY, USA: Association for Computing Machinery, Aug. 2016, pp. 1135–1144. ISBN: 978-1-4503-4232-2. DOI: [10.1145/2939672.2939778](https://doi.org/10.1145/2939672.2939778). URL: <https://dl.acm.org/doi/10.1145/2939672.2939778> (visited on 01/06/2026).
- [33] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser and Bjorn Ommer. ‘High-Resolution Image Synthesis with Latent Diffusion Models’. In: *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. New Orleans, LA, USA: IEEE, June 2022, pp. 10674–10685. ISBN: 978-1-6654-6946-3. DOI: [10.1109/CVPR52688.2022.01042](https://doi.org/10.1109/CVPR52688.2022.01042). URL: <https://ieeexplore.ieee.org/document/9878449/> (visited on 10/06/2026).
- [34] Cynthia Rudin. ‘Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead’. In: *Nature Machine Intelligence* 1.5 (May 2019), pp. 206–215. ISSN: 2522-5839. DOI: [10.1038/s42256-019-0048-x](https://doi.org/10.1038/s42256-019-0048-x). URL: <https://www.nature.com/articles/s42256-019-0048-x> (visited on 01/06/2026).
- [35] Mayu Sakurada and Takehisa Yairi. ‘Anomaly Detection Using Autoencoders with Nonlinear Dimensionality Reduction’. In: *Proceedings of the MLSDA 2014 2nd Workshop on Machine Learning for Sensory Data Analysis*. MLSDA’14. New York, NY, USA: Association for Computing Machinery, Dec. 2014, pp. 4–11. ISBN: 978-1-4503-3159-3. DOI: [10.1145/2689746.2689747](https://doi.org/10.1145/2689746.2689747). URL: <https://dl.acm.org/doi/10.1145/2689746.2689747> (visited on 05/06/2026).
- [36] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh and Dhruv Batra. ‘Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization’. In: *2017 IEEE International Conference on Computer Vision (ICCV)*. Oct. 2017, pp. 618–626. DOI: [10.1109/ICCV.2017.74](https://doi.org/10.1109/ICCV.2017.74). URL: <https://ieeexplore.ieee.org/document/8237336> (visited on 01/06/2026).

- [37] Irhum Shafkat. *Intuitively Understanding Variational Autoencoders*. Feb. 2018. URL: <https://medium.com/data-science/intuitively-understanding-variational-autoencoders-1bfe67eb5daf> (visited on 14/05/2026).
- [38] Karen Simonyan, Andrea Vedaldi and Andrew Zisserman. *Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps*. Apr. 2014. DOI: [10.48550/arXiv.1312.6034](https://doi.org/10.48550/arXiv.1312.6034). arXiv: [1312.6034](https://arxiv.org/abs/1312.6034) [cs.CV]. URL: <http://arxiv.org/abs/1312.6034> (visited on 11/06/2026).
- [39] Chung-En Sun, Tuomas Oikarinen, Berk Ustun and Tsui-Wei Weng. ‘Concept Bottleneck Large Language Models’. In: *International Conference on Learning Representations 2025* (May 2025), pp. 89371–89411. URL: https://proceedings.iclr.cc/paper_files/paper/2025/hash/de4ce91dfe56b919ee1c228d6a78f866-Abstract-Conference.html (visited on 01/06/2026).
- [40] Adly Templeton et al. *Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet*. URL: <https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html#appendix-citation> (visited on 20/10/2025).
- [41] Robert Tibshirani. ‘Regression Shrinkage and Selection Via the Lasso’. In: *Journal of the Royal Statistical Society: Series B (Methodological)* 58.1 (Jan. 1996), pp. 267–288. ISSN: 0035-9246. DOI: [10.1111/j.2517-6161.1996.tb02080.x](https://doi.org/10.1111/j.2517-6161.1996.tb02080.x). URL: <https://doi.org/10.1111/j.2517-6161.1996.tb02080.x> (visited on 29/05/2026).
- [42] Francesco Tonolini, Bjørn Sand Jensen and Roderick Murray-Smith. ‘Variational Sparse Coding’. In: *Proceedings of The 35th Uncertainty in Artificial Intelligence Conference*. PMLR, Aug. 2020, pp. 690–700. URL: <https://proceedings.mlr.press/v115/tonolini20a.html> (visited on 26/03/2026).
- [43] Aaron van den Oord, Oriol Vinyals and Koray Kavukcuoglu. ‘Neural Discrete Representation Learning’. In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., 2017. URL: <https://papers.nips.cc/paper/2017/hash/7a98af17e63a0ac09ce2e96d03992fbc-Abstract.html> (visited on 11/06/2026).
- [44] Pascal Vincent, Hugo Larochelle, Yoshua Bengio and Pierre-Antoine Manzagol. ‘Extracting and Composing Robust Features with Denoising Autoencoders’. In: *Proceedings of the 25th International Conference on Machine Learning - ICML ’08*. Helsinki, Finland: ACM Press, 2008, pp. 1096–1103. ISBN: 978-1-60558-205-4. DOI: [10.1145/1390156.1390294](https://doi.org/10.1145/1390156.1390294). URL: <http://portal.acm.org/citation.cfm?doid=1390156.1390294> (visited on 05/06/2026).

- [45] Sandra Wachter, Brent Mittelstadt and Chris Russell. *Counterfactual Explanations Without Opening the Black Box: Automated Decisions and the GDPR*. SSRN Scholarly Paper. Rochester, NY, Oct. 2017. DOI: [10.2139/ssrn.3063289](https://doi.org/10.2139/ssrn.3063289). Social Science Research Network: [3063289](https://papers.ssrn.com/abstract=3063289). URL: <https://papers.ssrn.com/abstract=3063289> (visited on 11/06/2026).
- [46] Lilian Weng. *From Autoencoder to Beta-VAE*. Aug. 2018. URL: <https://lilianweng.github.io/posts/2018-08-12-vae/> (visited on 05/06/2026).
- [47] Benjamin Wright and Lee Sharkey. ‘Addressing Feature Suppression in SAEs — AI Alignment Forum’. In: (Feb. 2024). URL: <https://www.alignmentforum.org/posts/3JuSjTZyMzaSeTxKk/addressing-feature-suppression-in-saes> (visited on 29/05/2026).

COLOPHON

This document was typeset using the typographical look-and-feel `classicthesis` developed by André Miede and Ivo Pletikosić. The style was inspired by Robert Bringhurst’s seminal book on typography “*The Elements of Typographic Style*”. `classicthesis` is available for both L^AT_EX and L^yX:

<https://bitbucket.org/amiede/classicthesis/>

Happy users of `classicthesis` usually send a real postcard to the author, a collection of postcards received so far is featured here:

<http://postcards.miede.de/>

Thank you very much for your feedback and contribution.